Lists and Loops
COS 125 Lab 6

Introduction:
Demonstrate your working code to the TA or MLA(s) before leaving lab sessions early. Work consistently throughout lab, ask questions, and submit what you have completed at the end of lab.

Submit what you have completed before leaving lab. If you want to complete more outside of lab, that's great for your growth, but submit what you have done before leaving. Work continuously during class, being off topic may result in lower grades. **Do not use AI or external-to-course resources.**

- Open VSCode or your preferred IDE
- Create a new folder named Lab6
- Inside the **Lab6** folder, create a python file named ***your_lastname_lab6.py***

Concepts:
Lists are ordered, heterogenous, indexed, contiguous in memory, comma separated, encapsulated by [ ] square brackets, complex data types. The elements in a list can be accessed using index. A list index is demonstrated by using `listName[index]` and the index designator is of integer type. The value inside the square brackets can be a integer literal (1, 2, 6) or a variable of type *int*.

Lists can be initialized as empty list = [], or populated with values. They can hold a mixture of the primitive data types (`int, str, bool, float, None`) and can hold complex data types (list, tuple).

Nesting lists (and tuples) creates the need to use an additional index indicator if when trying to step through the data held inside the lists.
Example:

```
myList = [ [ 11, 22, 33, 44 ],  [21, 31, 41, 51] ]
print(myList[0])
>> [ 11, 22, 33, 44 ]

print(myList[0][1])
>> 22
```

This structure lends itself nicely to using nested `for` loops, `len`(), and `range`() to step through elements in a nested list.

Slicing. Loosely this means: Using multiple indexes to indicate where to start and stop a selection. Slicing works similarly to range() in that it uses start and upToButNotIncludingStop indicators. The new element is the : colon to separate what indexes to start and stop at. This works with any list. If it's a list of lists the slicing gets more complex, as you have to get the list you want, then slice it.

Slicing also works on strings. If it has an index, you can designate what "slice" or selection of it you want to access with based on [start:exclusiveStop] and assigning that slice to a variable or other useful process.

```
myList2 = ['this', 'that', 'other', 'those', 'these']
print(myList2[1:4])
>> ['that', 'other', 'those']
```

Using the `.append()` method we can add a new element onto the end of an existing list.

```
otherList = []
otherList.append('hi')
print(otherList)
>> 'hi'
```

Creating a list from a string can be done using `.split()` method. Attaching the `.split()` to a string (which can also be attached to the end of `input()` since the datatype returned is str) will allow the program to separate (split) the values of the string into list elements based on the character(s) indicated in the ( ) of the split. Split returns a list of strings.

```
sentence = "This is a sentence."
sentList = sentence.split(" ")
print(sentList)
>>["This", "is", "a", " sentence."]
```

Challenge 1:

1. **Dataset Creation:**
    - Ask the user for a positive integer, which will represent the number of data points.
    - Create a list called number_list.
    - Demonstrate use of loops *for and while:* Using a loop prompt the user to enter as many integer values as the user gave as a positive number, one at a time, and add each integer to number_list.
        - Do this with a for loop.
        - Do this with a while loop.

2. **Dataset Creation with Split.**
    - Ask the user to enter a list of numbers separated by a comma and implement the .split(',') method. The new list should be named data_list.
    - Loop through the resulting list and convert to integers, storing the value back into the same index of the list that it was retrieved from. (e.g. if "6" was in index 4 then 6 should be saved back to index 4 of the list).

3. **Data Analysis:**
    - Using a **for** loop: Calculate the **sum** and **average** of all the numbers in the data_list without using the built-in sum() function.
    - Using a **while** loop: calculate the sum and average of all the numbers in the data_list without using the built-in sum() function.
    - Find the **minimum** and **maximum** values in the list without using built-in functions like min() or max(). You must use a loop and conditional logic,

you may choose which loop to use – this is to help understand what min() and max() are doing.

4. **Data Categorization:**
   - Initialize two new lists: even_numbers and odd_numbers.
   - Iterate through the data_list from above and append each number to the appropriate new list based on whether it is even or odd.
   - Count how many numbers are in each new list, you may use a built-in function.

5. **Display Results:**
   - Print the original data_list.
   - Print the calculated sum and average.
   - Print the minimum and maximum values or the even and odd lists.
   - Print the even_numbers and odd_numbers lists, along with their respective number in each list.

Example Output: (same content, doesn't need to look exactly the same)

```
Enter the number of data points: 5
Enter integer #1: 5
Enter integer #2: 1
Enter integer #3: 22
Enter integer #4: 20
Enter integer #5: 19

Enter numbers separated by commas: 12,3,25,8,15

Original Data: [12, 3, 25, 8, 15]
Sum: 63
Average: 12.6
Minimum Value: 3
Maximum Value: 25

Even Numbers: [12, 8] (Count: 2)
Odd Numbers: [3, 25, 15] (Count: 3)
```

Challenge 2: lab experiment data

The goal is to extract specific pieces of data from a mock scientific record using precise indexing and slicing techniques. **You should not use loops or functions for this exercise.**

Each record follows the content format:
["Subject ID", "Test Group", List of 4 Hourly Measurements (list of int)].

Copy and paste the data into your Python Code:

```
EXPERIMENT_RESULTS = [
    ['A-101', 'Control', [15, 18, 22, 25]],
    ['B-205', 'Variant', [12, 14, 17, 20]],
    ['C-330', 'Control', [16, 19, 23, 27]],
    ['D-410', 'Variant', [10, 11, 13, 15]]
    ]
```

Complete the following tasks by writing and executing a single line of Python code for each step that demonstrates **slicing** and use of **index.** Print the results.

Task 1: Basic Slicing

1. **Extract the records** for the two subjects in the **Variant** group (B-205 and D-410).
    - *Hint:* Use list indexing to select the specific records from the main list.
    - Store the result in a variable named variant_groups.
2. **Extract the last three hourly measurements** for the first subject (A-101).
    - *Hint:* This requires **nested access** (indexing into the record, then indexing into the measurements list) followed by **slicing**.
    - Store the result in a variable named a101_late_measurements.

Task 2: Indexed Slicing

1. **Extract the Subject ID and the final measurement (hour 4)** for the third subject (C-330). The result should be a list containing two elements.
    - *Hint:* Use **indexing** to pull the ID and **nested indexing** (perhaps using a negative index) to pull the final measurement.
    - Store the result in a variable named c330_summary.
2. **Extract only the second and third hourly measurements** for the second subject (B-205). The result should be the list [14, 17].
    - *Hint:* Use **nested access** to get to the measurements list, then apply **slicing** ([start:end]).
    - Store the result in a variable named b205_mid_test_readings.

Task 3: Complex Indexing

1. **Extract the Test Group** for the second subject (B-205) and the **Test Group** for the fourth subject (D-410), and store them together as a new list. The result should be ['Variant', 'Variant'].
    - *Hint:* This requires two separate **nested indexing** operations combined within a new list [...].
    - Store the result in a variable named all_variant_groups.

When all challenges are done, or the lab time ends submit to BrightSpace. If leaving early let the instructor know before you leave. Submit whatever you have completed before leaving. If you wish to work on the labs to further your own understanding after the lab ends, this is encouraged but be sure to submit what you did during lab.

When done, submit to BrightSpace and let the instructor know before you leave.